

PRIME CHECKING (trial division)

```
bool isPrime(long long n) {
    if (n < 2) return false;
    for (long long i=2; i*i<=n; i++) if (n%i==0) return false;
    return true;
}
```

Demo: count primes up to n (small n, no sieve)

```
int count=0; for(int i=2;i<=n;i++) if (isPrime(i)) count++;
// O(sqrt n) per check – fine for single queries. For n>=1e6+ or many
// queries, use Sieve of Eratosthenes (Rating 1200) instead.
```

MODULAR ARITHMETIC

// Always use long long before taking % to avoid overflow.

The three most common operations

```
(a + b) % MOD;
(a - b + MOD) % MOD; // add MOD to keep result non-negative
(a * b) % MOD;
const long long MOD = 1e9 + 7;
long long addMod(long long a, long long b) { return ((a%MOD)+(b%MOD))%MOD; }
long long subMod(long long a, long long b) { return ((a%MOD)-(b%MOD)+MOD)%MOD; }
long long mulMod(long long a, long long b) { return ((a%MOD)*(b%MOD))%MOD; }
```

Modular exponentiation (fast power) – $a^b \bmod MOD$

```
long long powMod(long long a, long long b, long long mod) {
    long long res=1; a%=mod;
    while (b > 0) {
        if (b & 1) res = res*a%mod;
        a = a*a%mod; b >>= 1;
    }
    return res;
}
```

Demo: modular inverse via Fermat's little theorem (mod must be prime)

```
long long modInverse(long long a, long long mod) { return powMod(a, mod-2, mod); }
// (a / b) % mod == (a * modInverse(b, mod)) % mod
```

Demo: compute $n! \bmod p$ for large n

```
long long fact=1; for(int i=1;i<=n;i++) fact=fact*i%MOD;
```

===== WHY DIVISION IS DIFFERENT UNDER MOD =====

```
(a / b) % MOD; // WRONG – division doesn't distribute over mod
(a * modInverse(b, MOD)) % MOD; //CORRECT – must multiply by modular inverse instead
```

===== MORE USEFUL MOD/PRIME NOTES =====

// modular exponentiation with negative exponent invalid, must use modInverse instead
// powMod(a, -1, mod) is WRONG – use modInverse(a, mod) for a^{-1}

// checking gcd(a, mod) == 1 before using modInverse (inverse only exists if coprime)

```
bool hasInverse = (__gcd(a, MOD) == 1);
// binomial coefficient C(n, r) mod p – needs factorials + modInverse
long long nCr(int n, int r, vector<long long>& fact, long long mod) {
    if (r < 0 || r > n) return 0;
    return fact[n] * modInverse(fact[r], mod) % mod * modInverse(fact[n-r], mod) %
mod;
}
```

// precompute factorials up to N once, reuse for many nCr queries

```
vector<long long> fact(N+1); fact[0]=1;
for (int i=1;i<=N;i++) fact[i] = fact[i-1]*i%MOD;
```